Ενότητα 06:
The Pipeline II

ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ

andreas.diavastos@cut.ac.cy

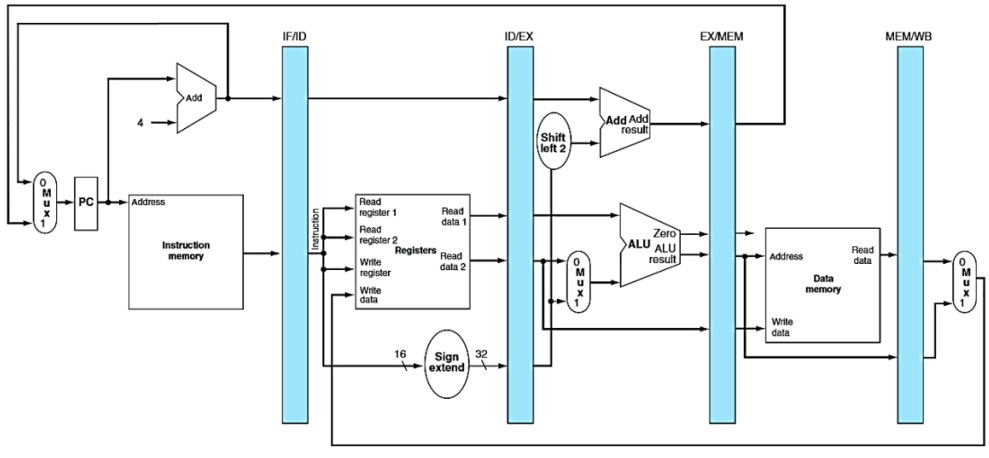
Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

CARL

1

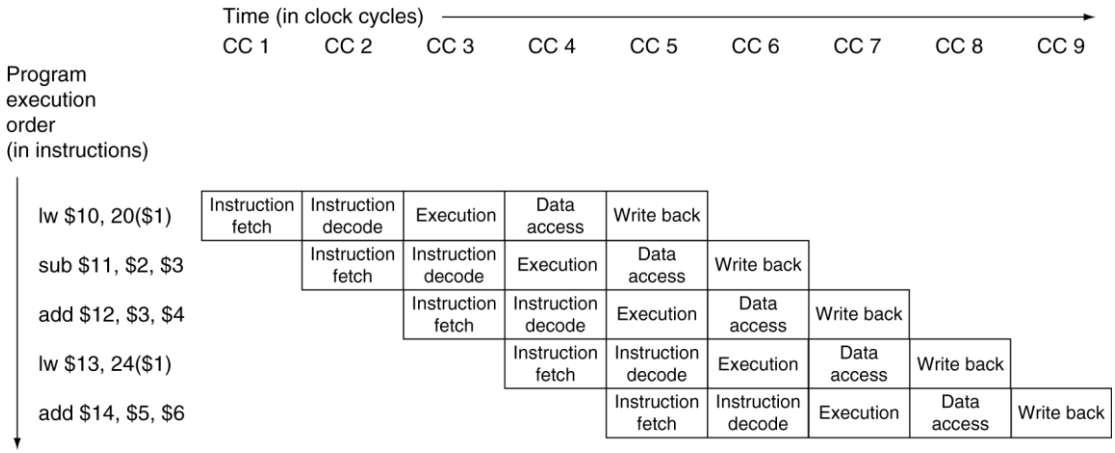
Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

Pipeline Datapath

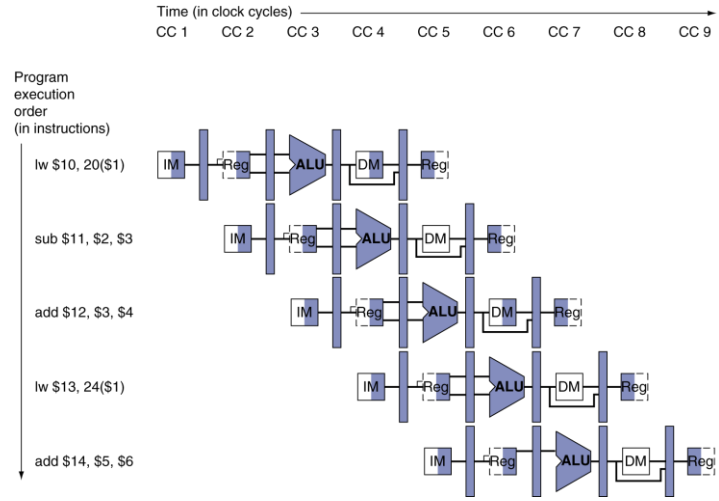


2

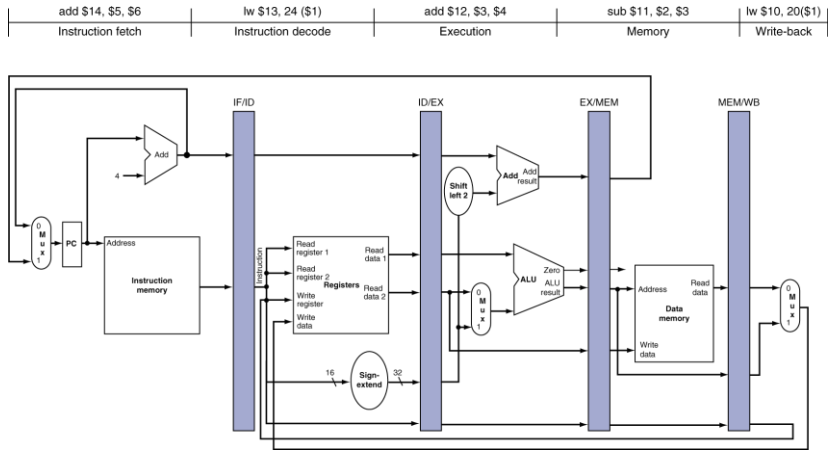
Multi-Cycle Pipeline Diagram (Traditional form)



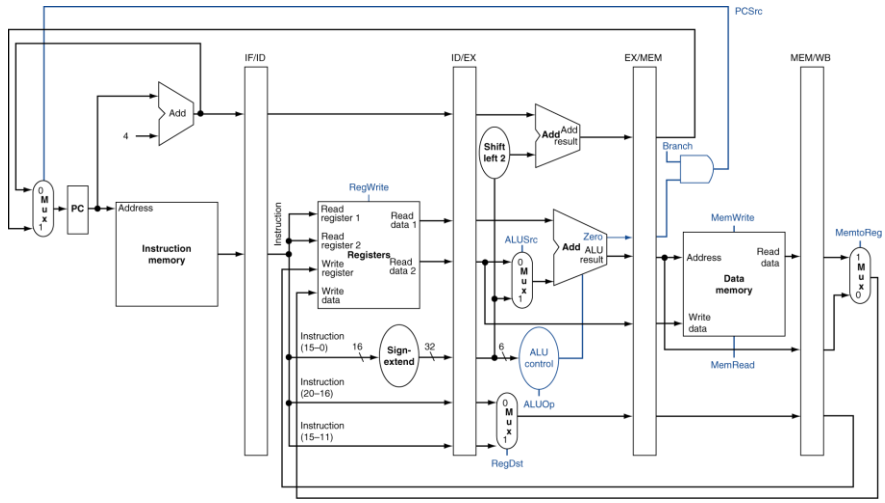
Multi-Cycle Pipeline Diagram (Resource usage)



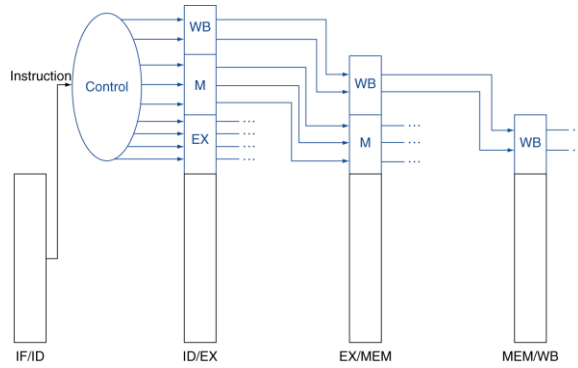
Pipeline Diagram 1-cycle Snapshot (State of pipeline in a given cycle)



Pipelined Control (Απλοποιημένο)

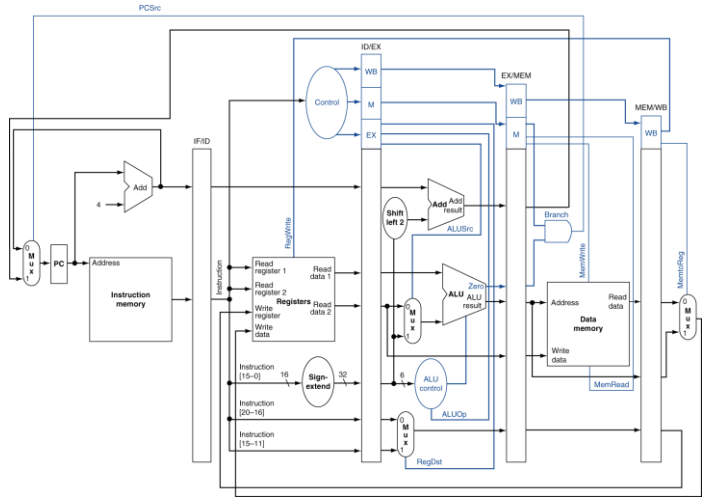


Pipelined Control



- Τα σήματα τα παίρνουμε από την εντολή όπως και στις single-cycle υλοποιήσεις του επεξεργαστή (**Decode Stage**)

Pipelined Control



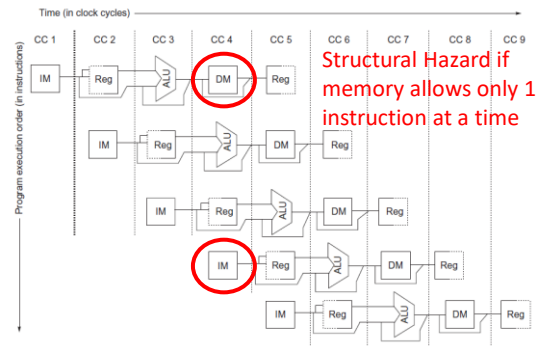
Pipeline Hazards

- Ιδεατό σενάριο: Το Pipeline εργάζεται χωρίς καθυστερήσεις, εάν κάθε εντολή είναι ανεξάρτητη από κάθε άλλη εντολή
- Στην πράξη υπάρχουν αλληλο-εξαρτήσεις μεταξύ εντολών ή/και μονάδων του επεξεργαστή
- **Κίνδυνοι Pipeline:** Εμποδίζουν την ομαλή και ανεξάρτητη λειτουργία του κάθε σταδίου μέσα στο pipeline και πιθανόν να προκαλέσουν καθυστερήσεις (stalling the pipeline)
- Τύποι κινδύνων Pipeline:
  - **Structural Hazards:** Δεν υπάρχουν αρκετά resources (μονάδες λειτουργιών)
  - **Data Hazards:** Εξάρτηση δεδομένων μεταξύ εντολών
  - **Control Hazards:** Αλλαγή κατεύθυνσης της εκτέλεσης των εντολών

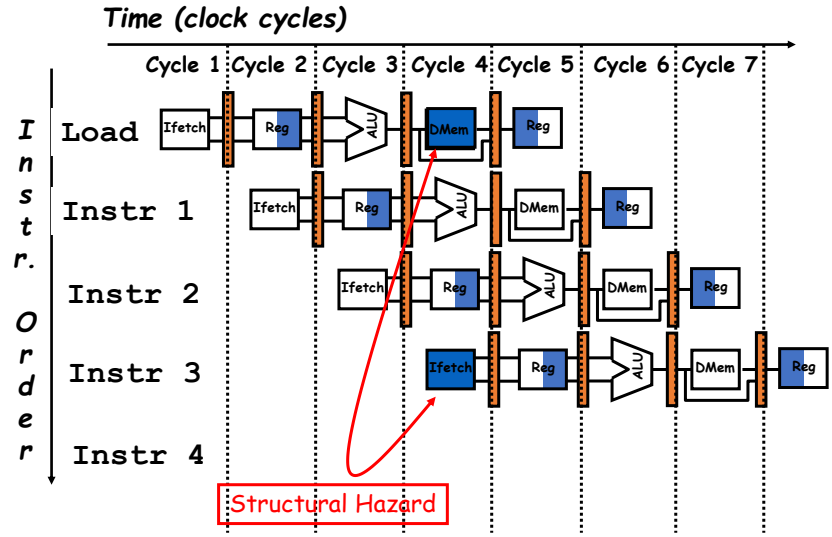
Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

Structural Hazards

- Συμβαίνουν όταν δύο ή περισσότερες εντολές σε διαφορετικά στάδια του pipeline απαιτούν τον ίδιο πόρο υλικού (resource) ταυτόχρονα
- Προκαλεί καθυστέρηση γιατί πρέπει να περιμένει η 2η εντολή
- Λύση: Περισσότερα resources ή Stalling (bubbles)

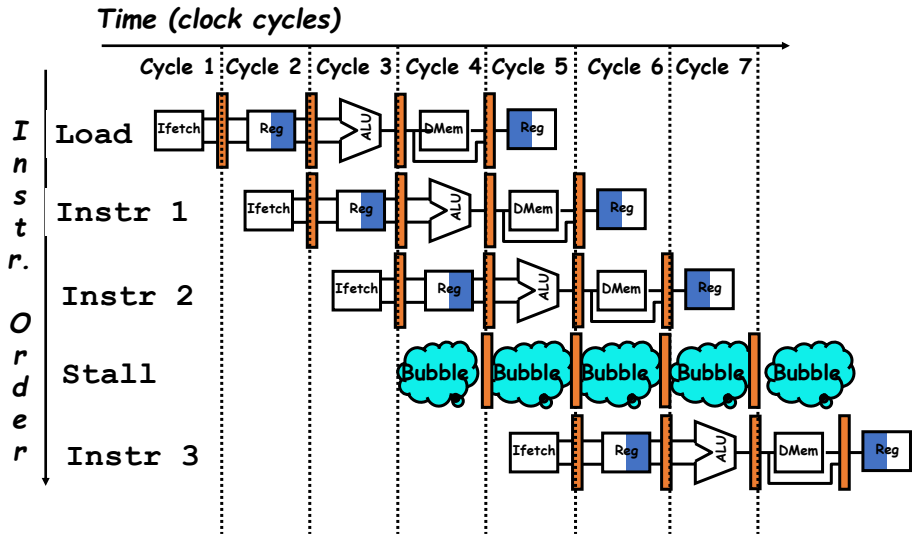


Παράδειγμα Structural Hazards



11

Παράδειγμα Structural Hazards – **Bubble Stall**



12

Data Hazards

- Προκύπτει όταν μια εντολή εξαρτάται από το αποτέλεσμα μιας προηγούμενης εντολής που δεν έχει ακόμη ολοκληρωθεί
- Τύποι:
  - **Read after Write (RAW)** (aka True Dependency) **:** Μια εντολή πρέπει να διαβάσει δεδομένα στα οποία γράφει μια προηγούμενη εντολή (ο πιο κοινός τύπος)

```
ADD R1, --, --;
SUB --, R1, --;
```


Data Hazards (συν.)

- **Write after Read (WAR)** (aka Anti-Dependency) **:** Συμβαίνει όταν μια εντολή γράφει σε έναν register από τον οποίο διαβάζει μια προηγούμενη εντολή. Συμβαίνει σε πιο μεγάλα pipeline που οι εντολές μπορεί να αλλάξουν σειρά εκτέλεσης.

```
ADD --, R1, --;
SUB R1, --, --;
```

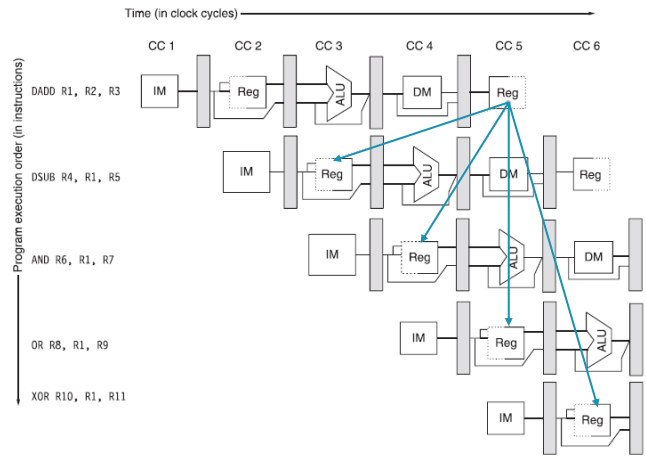
- **Write after Write (WAW)** (aka Output Dependency) **:** Εμφανίζεται όταν δύο εντολές γράφουν στον ίδιο register και η σειρά των εγγραφών πρέπει να διατηρηθεί

```
ADD R1, --, --;
SUB R1, --, --;
```

Παράδειγμα Data Hazards

```
add     x1,x2,x3
sub     x4,x1,x5
and     x6,x1,x7
or      x8,x1,x9
xor     x10,x1,x11
```

Read after Write (RAW)

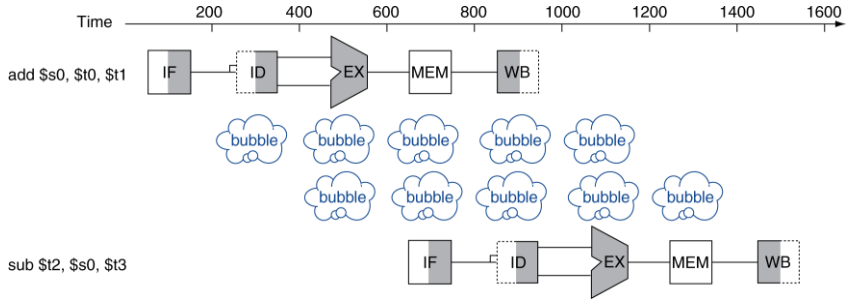


Απλή λύση για Data Hazards

- Μια εντολή εξαρτάται από (χρειάζεται) το αποτέλεσμα μίας προηγούμενης εντολής

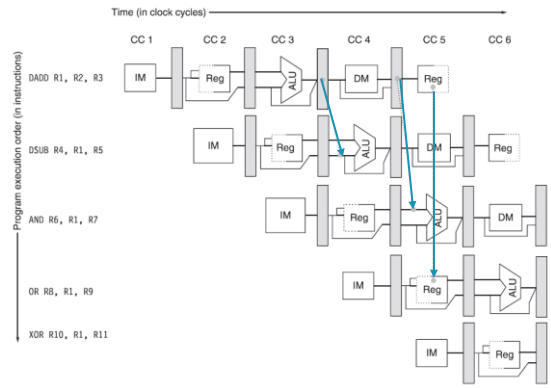
```
add $s0, $t0, $t1
sub $t2, $s0, $t3
```

Read after Write (RAW)



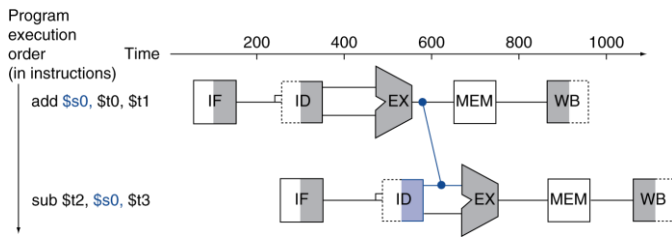
Data Hazards solutions: **Forwarding (Bypassing)**

- Μεταβιβάζουμε απευθείας το αποτέλεσμα από το ένα στάδιο στο άλλο, παρακάμπτοντας την ανάγκη αναμονής για την ολοκλήρωση της εντολής όλων των σταδίων
- Bypass network
- Αυξάνει την πολυπλοκότητα του επεξεργαστή αλλά είναι πολύ συνήθης τακτική!



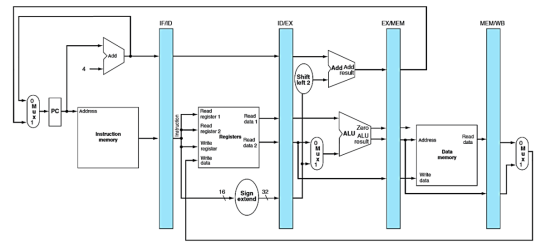
Forwarding (aka Bypassing)

- Χρησιμοποιούμε το αποτέλεσμα (εάν χρειάζεται) μόλις παραχθεί
  - Δεν περιμένουμε να γραφτεί πρώτα στον καταχωρητή
    - Δεν παραλείπουμε όμως το γράψιμο στο τελευταίο στάδιο. Αυτό θα γίνει ούτως ή αλλιώς
  - Χρειάζεται επιπλέον συνδέσεις μεταξύ μονάδων του επεξεργαστή



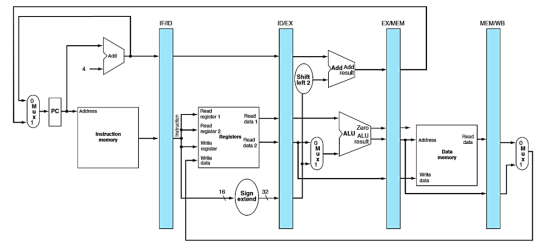
Detecting the Need to Forward

- Μεταφέρουμε τον αριθμό των registers μέσα από το pipeline
  - π.χ., ID/EX.RegisterRs = καταχωρητής για Rs που βρίσκεται στον ενδιάμεσο καταχωρητή ID/EX (pipeline register)
- Για την ALU, οι καταχωρητές εισόδου της πράξης δίνονται από:
  - ID/EX.RegisterRs
  - ID/EX.RegisterRt
- **Data Hazards όταν:**
  1. EX/MEM.RegisterRd = ID/EX.RegisterRs
  2. EX/MEM.RegisterRd = ID/EX.RegisterRt
  3. MEM/WB.RegisterRd = ID/EX.RegisterRs
  4. MEM/WB.RegisterRd = ID/EX.RegisterRt

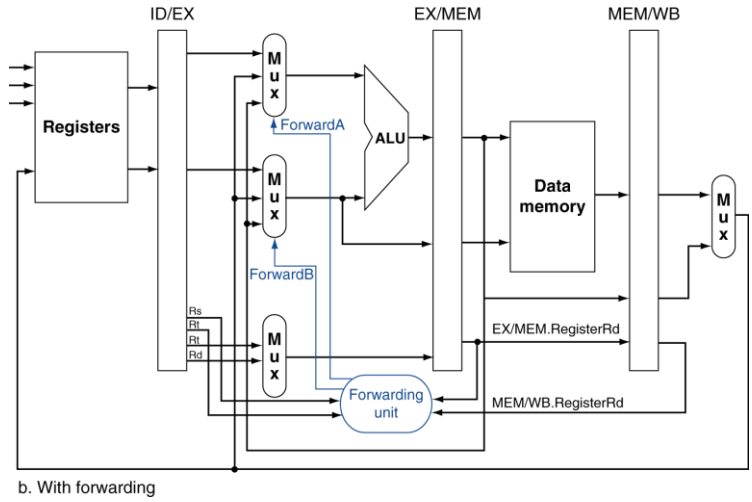


Detecting the Need to Forward

- Αλλά μόνο όταν η εντολή θα γράψει σε ένα καταχωρητή
  - EX/MEM.RegWrite
  - MEM/WB.RegWrite
- Και μόνο εάν ο Rd για εκείνη την εντολή δεν είναι ο $zero
  - EX/MEM.RegisterRd ≠ 0
  - MEM/WB.RegisterRd ≠ 0



Forwarding Paths



b. With forwarding

Ενότητα 06: The Pipeline II

ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ

andreas.diavastos@cut.ac.cy